Q21 Comprehension Check

Question: A colleague proposes a warehouse robot built as a 4-stage pipeline: object detector → grasp planner → motion controller → safety monitor. Each stage is a separately trained network. They report that each stage is "95% accurate in unit tests" and therefore expect roughly 95% end-to-end success. (a) What is wrong with the arithmetic, and under what assumption would 95% actually be the right expectation? (b) Name two composition costs their design incurs that a monolithic end-to-end policy would not. (c) Their next proposal is to make the stages talk by passing natural-language descriptions between them, "so we can read the logs." What does this interface choice cost them relative to structured embeddings?

Answer: (a) Simple multiplication of per-stage accuracies ($0.95^4 \approx 0.81$) is only correct if stage errors are independent; in practice errors correlate (hard inputs fail at multiple stages), so end-to-end success is usually *lower* than 0.81 — compounding error, not the unit-test product. (b) Any two of: compounding/propagated error across fixed seams with no way for downstream stages to correct upstream mistakes; interface brittleness (retraining the detector silently shifts the grasp planner's input distribution); cross-component credit assignment (a failed grasp cannot be localized to detector vs. planner without extra instrumentation); added serial latency. (c) Natural language is the lowest-bandwidth, lossiest interface: continuous geometry (poses, grasp points) cannot round-trip through text without quantization error, gradients cannot flow across the seam for joint fine-tuning, and generation adds latency — they gain log readability (observability) but pay in accuracy ceiling and trainability of the joints.